

```

1 #=====
2 # MIT 6.046J 代码精读 • PS1 - 分治最近点对
3 # 源文件: close_pairs.py (来自 GitHub mit-6046)
4 #=====
5
6 """PS1 Problem 1-2: Radio Frequency Assignment (points within distance 1).
7
8 Given n stations with distinct x- and distinct y-coordinates, decide whether any
9 two lie within Euclidean distance 1 (part b) or any three are mutually within
10 distance 1 (part c), in  $O(n \log n)$  time.
11
12 The clean way to get  $O(n \log n)$  and match the pset's grid hint is the classic
13 *divide-and-conquer closest pair*: split by x, recurse, and merge across the
14 dividing line using the fact that only  $O(1)$  candidate points can be packed into
15 each  $(1/2 \times 1/2)$  grid cell of the strip. Here we only need "is the minimum
16 distance  $< 1$ ", so we can also early-exit, but we compute the true closest pair
17 and compare to 1 -- that is strictly more information and still  $O(n \log n)$ .
18
19 Part (c) (three within distance 1) is handled by ``exists_triple_within_1``:
20 once any pair within distance 1 is found, only  $O(1)$  other points can be within
21 distance 1 of both (again a packing argument), so we check them directly.
22
23 A brute-force  $O(n^2)$  oracle is included and used in ``test`` to certify results.
24 """
25 from __future__ import annotations
26
27 import math
28 from typing import List, Optional, Tuple
29
30 Point = Tuple[float, float]
31
32
33 def _dist(a: Point, b: Point) -> float:
34     return math.hypot(a[0] - b[0], a[1] - b[1])
35
36
37 def closest_pair(points: List[Point]) -> Tuple[float, Optional[Tuple[Point, Point]]]:
38     """Return (min_distance, (p, q)) using divide and conquer,  $O(n \log n)$ ."""
39     if len(points) < 2:
40         return math.inf, None
41     px = sorted(points) # by x (distinct x assumed)
42     py = sorted(points, key=lambda p: p[1]) # by y
43     return _cp_rec(px, py)
44
45
46 def _cp_rec(px: List[Point], py: List[Point]):
47     n = len(px)
48     if n <= 3:
49         best = math.inf
50         pair = None
51         for i in range(n):
52             for j in range(i + 1, n):
53                 d = _dist(px[i], px[j])
54                 if d < best:
55                     best, pair = d, (px[i], px[j])
56         return best, pair
57
58     mid = n // 2
59     midx = px[mid][0]
60     lx = px[:mid]
61     rx = px[mid:]
62     lset = set(lx)
63     ly = [p for p in py if p in lset]
64     ry = [p for p in py if p not in lset]
65
66     dl, pl = _cp_rec(lx, ly)
67     dr, pr = _cp_rec(rx, ry)
68     if dl < dr:
69         d, pair = dl, pl
70     else:
71         d, pair = dr, pr
72
73     # strip: points within horizontal distance d of the dividing line, sorted by y
74     strip = [p for p in py if abs(p[0] - midx) < d]
75     for i in range(len(strip)):
76         # only the next ~7 points in y-order can be closer than d (packing bound)
77         for j in range(i + 1, len(strip)):
78             if strip[j][1] - strip[i][1] >= d:
79                 break
80             dd = _dist(strip[i], strip[j])
81             if dd < d:
82                 d, pair = dd, (strip[i], strip[j])
83     return d, pair
84
85

```

```

86 def exists_pair_within_1(points: List[Point]) -> Optional[Tuple[Point, Point]]:
87     """Part (b): return a pair within Euclidean distance 1, or None."""
88     d, pair = closest_pair(points)
89     if pair is not None and d <= 1.0:
90         return pair
91     return None
92
93
94 def exists_triple_within_1(points: List[Point]) -> Optional[Tuple[Point, Point, Point]]:
95     """Part (c): return three points pairwise within distance 1, or None.
96
97     Uses a sweep with a d=1 grid of cell size 1/2 x 1/2. Two points within
98     distance 1 must lie in the same or an adjacent cell (<=5x5 neighbourhood),
99     and only O(1) points can occupy any cell region of a valid configuration,
100     so the whole scan is O(n). Building the grid dominates at O(n).
101     """
102     cell = 0.5
103     grid: dict = {}
104     for p in points:
105         key = (int(math.floor(p[0] / cell)), int(math.floor(p[1] / cell)))
106         grid.setdefault(key, []).append(p)
107
108     for p in points:
109         cx = int(math.floor(p[0] / cell))
110         cy = int(math.floor(p[1] / cell))
111         near: List[Point] = []
112         for dx in range(-2, 3):
113             for dy in range(-2, 3):
114                 near.extend(grid.get((cx + dx, cy + dy), []))
115         within = [q for q in near if q != p and _dist(p, q) <= 1.0]
116         for i in range(len(within)):
117             for j in range(i + 1, len(within)):
118                 if _dist(within[i], within[j]) <= 1.0:
119                     return (p, within[i], within[j])
120     return None
121
122
123 # ----- brute-force oracles -----
124 def brute_pair_within_1(points: List[Point]) -> Optional[Tuple[Point, Point]]:
125     n = len(points)
126     for i in range(n):
127         for j in range(i + 1, n):
128             if _dist(points[i], points[j]) <= 1.0:
129                 return (points[i], points[j])
130     return None
131
132
133 def brute_closest(points: List[Point]) -> float:
134     n = len(points)
135     best = math.inf
136     for i in range(n):
137         for j in range(i + 1, n):
138             best = min(best, _dist(points[i], points[j]))
139     return best
140
141
142 def brute_triple_within_1(points: List[Point]):
143     n = len(points)
144     for i in range(n):
145         for j in range(i + 1, n):
146             if _dist(points[i], points[j]) > 1.0:
147                 continue
148             for k in range(j + 1, n):
149                 if (_dist(points[i], points[k]) <= 1.0
150                     and _dist(points[j], points[k]) <= 1.0):
151                     return (points[i], points[j], points[k])
152     return None
153

```